



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Department of Information Science and Engineering

Project Based Learning Report on Smart Helmet with Fall Detection and Alert System for the course IoT and Applications (IS244AI)

Submitted by

Sanath Reddy

1RV24IS109

Sathi Surya Prakash Reddy

1RV24IS114

Under the guidance of

Prof. B K Srinivas

In partial fulfilment for the award of degree of

Bachelor of Engineering

in

Information Science Engineering

RV COLLEGE OF ENGINEERING®

BENGALURU-59

(Autonomous Institution Affiliated to VTU, Belagavi)

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING



CERTIFICATE

Certified that the project work titled “**Smart Helmet with Fall Detection and Alert System**” is carried out by **SANATH REDDY (1RV24IS109)**, **SATHI SURYA PRAKASH REDDY (1RV24IS114)** in partial fulfilment of the completion of the course **IoT and Applications (IS244AI)** of the IV Semester, B.E., in **Information Science and Engineering**, during the academic year 2025-26. It is certified that all corrections/suggestions indicated for the Laboratory Internal Assessment have been incorporated in the project report and duly approved by the faculty. The project report has been approved as it satisfies the academic requirements in respect of internship project work prescribed by the institution for the said degree.

Signature of Faculty In-charge

Signature of HoD

External Viva

Name of Examiner

Signature with Date

1.

2.

Abstract of Smart Helmet with Fall Detection and Alert System

Road accidents involving two-wheeler riders are a major cause of injuries and fatalities worldwide. In many cases, delayed medical assistance after an accident significantly increases the severity of injuries and reduces the chances of survival. To address this issue, this project presents a Smart Helmet with Fall Detection and Alert System, an IoT-based rider safety solution designed to monitor rider conditions, detect accident events in real time, and automatically notify emergency contacts with the rider's location.

The proposed system is built around an ESP32 microcontroller integrated with multiple sensors, including an MPU6050 accelerometer and gyroscope, a NEO-6M GPS module, and an MQ135 gas and alcohol sensor. The MPU6050 continuously monitors acceleration and motion parameters to identify sudden impacts and fall events. When the measured acceleration exceeds a predefined threshold, the system classifies the event as a potential accident and activates an emergency alert procedure. The GPS module provides real-time geographical coordinates, enabling accurate location tracking of the rider. Additionally, the MQ135 sensor monitors environmental gas concentration and can indicate the presence of alcohol-related vapors, contributing to rider safety awareness.

The helmet communicates wirelessly with a backend server through Wi-Fi using WebSocket technology. The backend, developed using Node.js and Express.js, receives telemetry data from the helmet, manages real-time communication with connected clients, and processes emergency notifications. Upon accident detection, the server automatically generates and sends SMS alerts through the Twilio communication platform, containing the accident warning, GPS coordinates, and a Google Maps link to the rider's location. This automated response mechanism helps reduce the time required to inform family members, emergency responders, or nearby assistance services.

A web-based dashboard is also developed to provide real-time visualization of rider telemetry. The dashboard displays live location tracking on an interactive map, acceleration data from the IMU sensors, gas concentration levels, and accident status indicators. Audio and visual alerts are generated whenever an accident is detected, enabling operators to monitor rider safety remotely. The system also supports telemetry simulation and testing features, allowing functionality verification even when physical hardware is unavailable.

To improve reliability, the proposed solution incorporates fault-tolerant mechanisms such as GPS staleness detection, offline telemetry buffering, automatic reconnection with exponential backoff, and accident-state hysteresis. These features ensure continuous operation under varying network and environmental conditions.

The Smart Helmet with Fall Detection and Alert System demonstrates how IoT technologies, wireless communication, and real-time monitoring can be integrated into a practical safety device for motorcyclists. By combining accident detection, location tracking, emergency alerting, and remote monitoring into a single platform, the system aims to enhance rider safety, reduce emergency response times, and contribute to the development of intelligent transportation safety solutions.

Table of Contents

Chapter	Title	Page No.
1	Introduction to “Smart Helmet with Fall Detection and Alert System”	1
2	Existing system study and its outcome with project objectives	4
3	Project architecture & design (circuit diagrams, pin configurations)	6
4	Methodology for “Smart Helmet with Fall Detection and Alert System”	12
5	Results of “Smart Helmet with Fall Detection and Alert System”	19
Appendix	Code, Acronyms	25

CHAPTER 1

INTRODUCTION

1.1 Background

Road transportation plays a vital role in modern society, with motorcycles being one of the most widely used modes of personal transportation due to their affordability, fuel efficiency, and convenience. However, two-wheeler riders are highly vulnerable to road accidents because they lack the physical protection available in enclosed vehicles. According to various road safety studies, a significant percentage of fatalities occur not only because of the accident itself but also due to delays in emergency response and medical assistance.

Helmets are mandatory safety equipment designed to reduce head injuries during collisions. While conventional helmets provide passive protection, they are unable to monitor the rider's condition, detect accidents, or communicate emergency information. With the rapid growth of the Internet of Things (IoT), embedded systems, and wireless communication technologies, it has become possible to transform traditional helmets into intelligent safety devices capable of monitoring rider activity and responding automatically during emergencies.

The Smart Helmet with Fall Detection and Alert System is designed to address these challenges by integrating sensing, communication, and monitoring technologies into a single platform. The system continuously monitors rider movement, detects falls or accidents, tracks geographical location, and automatically sends alerts to emergency contacts, thereby reducing response time and potentially saving lives.

1.2 Need for the System

Motorcycle accidents often occur in remote locations or during times when immediate assistance is unavailable. In many situations, unconscious or severely injured riders are unable to contact family members, hospitals, or emergency services. This delay in communication can significantly worsen the consequences of an accident.

Another challenge is the lack of real-time monitoring of rider safety conditions. Traditional helmets do not provide any information regarding accidents, rider location, or environmental conditions. Families and emergency responders often remain unaware of an accident until much later.

The proposed Smart Helmet with Fall Detection and Alert System addresses these issues by automatically detecting accident events, obtaining GPS coordinates, and notifying emergency contacts through SMS alerts. The integration of IoT technology ensures continuous monitoring and real-time communication, making the system an effective solution for enhancing rider safety.

1.3 Overview of the Proposed System

The proposed system is an IoT-enabled smart safety helmet built around an ESP32

microcontroller. The helmet incorporates multiple sensors that collect and transmit real-time data related to rider movement, location, and environmental conditions.

An MPU6050 accelerometer and gyroscope continuously monitor acceleration and motion parameters. When a sudden impact or abnormal fall is detected based on predefined thresholds, the system identifies the event as a potential accident. A NEO-6M GPS module provides real-time geographical coordinates, enabling accurate tracking of the rider's location. Additionally, an MQ135 gas sensor measures gas concentration levels and can indicate the presence of alcohol-related vapors.

The ESP32 processes sensor data and communicates with a backend server using Wi-Fi and WebSocket technology. Upon accident detection, the server automatically sends emergency SMS notifications containing the rider's location details and a Google Maps link. A web-based dashboard allows real-time visualization of rider telemetry, including location tracking, acceleration values, gas concentration, and accident status.

1.4 Internet of Things in Rider Safety

The Internet of Things refers to a network of interconnected devices capable of collecting, processing, and exchanging data over communication networks. IoT technology has transformed several sectors including healthcare, transportation, agriculture, and industrial automation.

In the field of transportation safety, IoT enables continuous monitoring of vehicles and riders through embedded sensors and communication systems. Smart helmets represent an important application of IoT, where sensor data can be analyzed in real time to identify hazardous situations and generate automatic responses.

The proposed system leverages IoT principles by integrating sensors, wireless communication, cloud-connected services, and remote monitoring interfaces. This combination enables real-time accident detection, emergency notification, and location tracking, thereby improving rider safety and reducing the risks associated with delayed medical intervention.

1.5 Objectives of the Project

The primary objective of the Smart Helmet with Fall Detection and Alert System is to improve the safety of motorcycle riders through intelligent monitoring and automated emergency response mechanisms.

The specific objectives are:

- To detect rider falls and accident events using accelerometer and gyroscope data.
- To monitor rider location using GPS technology.
- To provide real-time telemetry transmission through wireless communication.
- To automatically send emergency alerts containing accident location details.
- To develop a web dashboard for live monitoring and visualization of rider data.

- To reduce emergency response time and improve the chances of timely medical assistance.

1.6 Scope of the Project

The Smart Helmet with Fall Detection and Alert System is intended for motorcycle riders, delivery personnel, commuters, and individuals who frequently travel on two-wheelers. The system can be deployed in urban as well as rural environments where rapid accident reporting is essential.

The project focuses on accident detection, location tracking, emergency notification, and real-time monitoring through IoT technologies. The system can be further enhanced by integrating additional features such as heart-rate monitoring, machine learning-based accident prediction, cloud-based analytics, mobile applications, and direct communication with emergency services.

By combining embedded systems, wireless communication, and intelligent monitoring, the proposed solution contributes toward the development of safer and smarter transportation systems.

CHAPTER 2: EXISTING SYSTEM AND PROJECT OBJECTIVES

2.1 Existing System Study

Road safety has been a major concern due to the increasing number of motorcycle accidents worldwide. Traditional safety measures mainly rely on protective equipment such as helmets, riding gear, and traffic regulations. While these measures help reduce the severity of injuries, they do not provide any mechanism for detecting accidents or informing emergency services automatically.

Several accident detection systems have been proposed in recent years. Some systems use smartphone applications to detect sudden impacts and send alerts. However, these applications depend on the rider carrying the phone at all times and may fail due to battery depletion, network issues, or incorrect sensor readings. Other vehicle-based monitoring systems are installed directly on motorcycles, but they often increase system complexity and cost while lacking portability.

Conventional helmets provide only passive protection and do not possess the capability to monitor rider conditions, detect falls, track location, or communicate emergency information. As a result, accident victims may remain unattended for extended periods, especially in isolated areas where witnesses are unavailable. The delay in providing medical assistance can significantly increase the risk of severe injury or fatality.

Recent developments in the Internet of Things (IoT) have enabled the integration of sensors, wireless communication, and cloud-based monitoring systems into wearable safety devices. Smart helmets have emerged as a promising solution by combining accident detection, location tracking, and emergency notification features within a single platform. However, many existing solutions are limited by unreliable communication, insufficient monitoring capabilities, or lack of real-time visualization.

2.2 Outcome of Existing System Study

The study of existing systems revealed several limitations that affect their effectiveness in real-world accident scenarios. The major observations are:

- Traditional helmets provide protection but cannot detect accidents or communicate emergency information.
- Smartphone-based solutions depend heavily on user behavior and device availability.
- Many existing systems lack accurate real-time location tracking.
- Delays in emergency notification can increase the severity of accident outcomes.
- Limited monitoring and visualization capabilities make remote supervision difficult.
- Network interruptions may result in loss of critical accident information.
- Most systems do not provide a comprehensive platform integrating accident detection, tracking, alerting, and monitoring.

These findings indicate the need for an integrated, reliable, and intelligent safety system capable of providing immediate response during emergencies.

2.3 Project Objectives

Based on the identified limitations of existing systems, the Smart Helmet with Fall Detection and Alert System has been developed with the following objectives:

1. To continuously monitor rider movement using accelerometer and gyroscope sensors.
2. To accurately detect falls and accident events through real-time motion analysis.
3. To obtain the rider's geographical location using a GPS module.
4. To transmit telemetry data wirelessly using IoT communication technologies.
5. To automatically generate emergency alerts when an accident is detected.
6. To send SMS notifications containing accident location details to emergency contacts.
7. To provide a web-based dashboard for live monitoring and visualization of rider data.
8. To ensure reliable operation through automatic reconnection and offline data buffering mechanisms.
9. To reduce emergency response time and improve rider safety through automated assistance.

2.4 Proposed Solution

The proposed Smart Helmet with Fall Detection and Alert System integrates an ESP32 microcontroller, MPU6050 motion sensor, NEO-6M GPS module, and MQ135 gas sensor to create an intelligent rider safety platform. The system continuously monitors acceleration and motion data to identify fall events and accidents. Upon detecting an accident, the helmet automatically transmits location information to a backend server, which then sends emergency SMS alerts to predefined contacts.

A web-based monitoring dashboard provides real-time visualization of rider telemetry, including location, acceleration data, gas concentration levels, and accident status. The integration of IoT communication, automated alerting, and live monitoring makes the proposed system a comprehensive solution for improving motorcycle rider safety and reducing accident response time.

Chapter 3

PROJECT ARCHITECTURE & DESIGN

3.1 Introduction

The Smart Helmet with Fall Detection and Alert System is an Internet of Things (IoT)-based safety solution developed to improve the protection of motorcyclists during road travel. Accidents involving two-wheelers often result in severe injuries due to delayed emergency response. The proposed system addresses this challenge by continuously monitoring rider conditions, detecting fall events in real time, determining the rider's geographical location, and automatically generating emergency alerts.

The system combines multiple hardware and software components to create an integrated safety platform. An ESP32 microcontroller acts as the central processing unit and interfaces with various sensors to collect real-time information. The collected data is processed locally and transmitted to a backend server through wireless communication. The server further analyzes the information, updates a monitoring dashboard, and sends emergency notifications when required.

The architecture has been designed with reliability, scalability, and real-time responsiveness in mind. Features such as automatic reconnection, data buffering, GPS validation, and accident-state latching ensure stable operation even under adverse conditions. The following sections describe the overall architecture, hardware design, circuit implementation, and pin configuration used in the system.

3.2 System Architecture

The overall architecture of the Smart Helmet with Fall Detection and Alert System consists of four major layers: the sensing layer, processing layer, communication layer, and monitoring & alert layer. Each layer performs a specific function and collectively contributes to the efficient operation of the system.

The sensing layer includes the MPU6050 motion sensor, NEO-6M GPS module, and MQ135 gas sensor. These sensors continuously gather information regarding rider movement, geographical location, and surrounding gas concentration levels. The processing layer consists of the ESP32 microcontroller, which acquires sensor data, performs accident detection calculations, manages communication protocols, and controls data transmission.

The communication layer establishes wireless connectivity between the helmet and the backend server through Wi-Fi and WebSocket protocols. This enables real-time transmission of telemetry data without significant delays. The monitoring and alert layer consists of the Node.js backend server, web dashboard, and emergency notification system. This layer is responsible for visualizing sensor data, maintaining communication with connected devices, and generating emergency alerts when accidents occur.

The architecture follows a modular design approach. Each component can operate independently while contributing to the overall functionality of the system. This

approach improves maintainability, simplifies troubleshooting, and enables future enhancements such as cloud integration, machine learning-based accident prediction, and advanced rider analytics.

SMART HELMET WITH FALL DETECTION AND ALERT SYSTEM – SYSTEM ARCHITECTURE

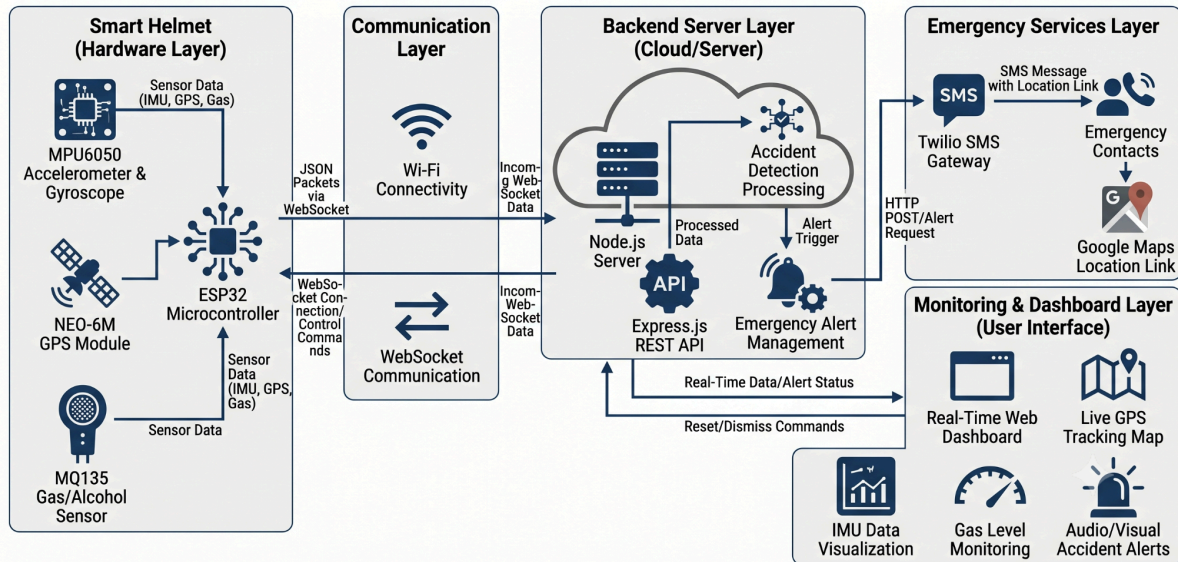


Figure 3.1: System Architecture of Smart Helmet with Fall Detection and Alert System

Components of the Architecture

ESP32 Controller

The ESP32 serves as the main controller of the system. It receives sensor data, processes measurements, performs accident detection calculations, and transmits telemetry information to the backend server. The ESP32 was selected due to its integrated Wi-Fi capability, dual-core processing architecture, low power consumption, and support for multiple communication interfaces.

MPU6050 Motion Sensor

The MPU6050 combines a three-axis accelerometer and a three-axis gyroscope in a single package. The accelerometer measures linear acceleration while the gyroscope measures angular velocity. These measurements are used to identify sudden impacts, falls, and abnormal rider movements.

NEO-6M GPS Module

The GPS module provides accurate location information by receiving signals from navigation satellites. Latitude, longitude, speed, and satellite lock information are continuously monitored and transmitted to the server. During accidents, GPS coordinates are included in emergency alerts to assist rescue teams in locating the rider.

MQ135 Gas Sensor

The MQ135 sensor monitors gas concentration levels in the surrounding environment.

It is capable of detecting gases such as alcohol vapors, ammonia, benzene, and smoke. The sensor contributes an additional safety feature by identifying potentially hazardous environmental conditions.

Backend Server

The Node.js server acts as a communication bridge between the helmet hardware and the monitoring dashboard. It receives telemetry data, manages WebSocket connections, processes alerts, and coordinates emergency notification services.

Web Dashboard

The dashboard provides a user-friendly interface for monitoring rider status in real time. It displays sensor readings, GPS location, accident alerts, and communication status. Interactive visualizations improve situational awareness and facilitate rapid decision-making during emergencies.

3.3 Circuit Diagram

The circuit diagram represents the electrical interconnections between the ESP32 microcontroller and the sensors used in the system. Proper circuit design is essential to ensure accurate sensor operation, stable communication, and reliable power distribution.

The MPU6050 sensor is connected through the I2C communication bus, which requires only two signal lines for data transfer. This simplifies wiring while enabling high-speed communication between the sensor and the ESP32. The GPS module communicates through UART serial communication, allowing continuous reception of location data. The MQ135 sensor produces an analog voltage corresponding to gas concentration levels, which is measured using the ESP32's built-in Analog-to-Digital Converter (ADC).

All modules share a common ground reference to maintain signal integrity. Separate voltage supplies are provided according to the operating requirements of individual sensors. The ESP32 receives processed information from each sensor and combines it into a unified telemetry stream for transmission.

The circuit design prioritizes reliability, low power consumption, and ease of implementation. The chosen configuration minimizes wiring complexity while ensuring stable communication between all components.

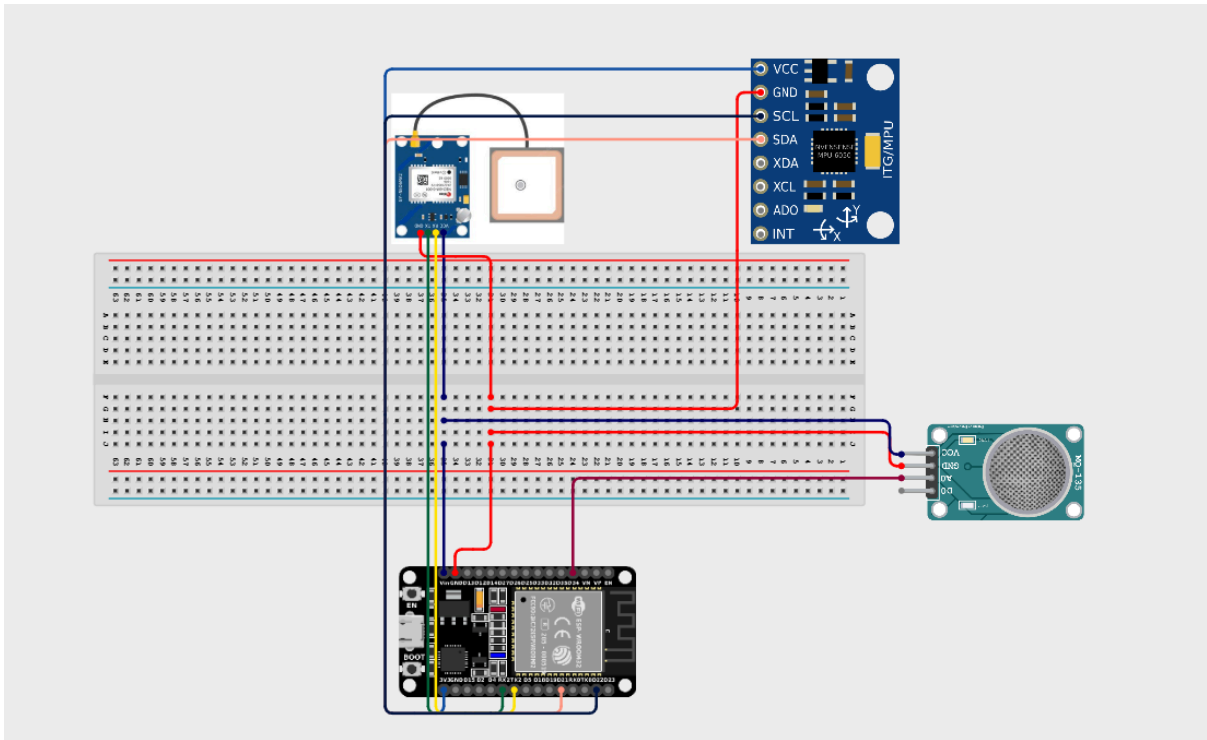


Figure 3.2: Circuit Diagram of Smart Helmet with Fall Detection and Alert System

Power Distribution

The ESP32 and MPU6050 operate at 3.3V logic levels, while the MQ135 sensor requires a 5V supply for proper heater operation. The GPS module can operate at either 3.3V or 5V depending on the module version. Proper voltage regulation ensures safe and reliable system operation.

Communication Interfaces

Three different communication methods are utilized within the system:

- I2C Communication for MPU6050 sensor data transfer.
- UART Communication for GPS data reception.
- Analog ADC Interface for MQ135 gas concentration measurement.

The use of multiple communication interfaces demonstrates the versatility of the ESP32 microcontroller and allows efficient integration of diverse sensing technologies.

3.4 Pin Configuration and GPIO Mapping

Pin configuration plays a critical role in ensuring successful communication between the ESP32 and connected peripherals. Each sensor is assigned dedicated GPIO pins based on hardware requirements and communication protocol compatibility.

The ESP32 supports multiple communication interfaces including I2C, UART, SPI, and ADC channels. Appropriate GPIO allocation reduces communication conflicts

and improves system performance.

MPU6050 Pin Configuration

The MPU6050 sensor utilizes the default ESP32 I2C interface. GPIO21 is configured as the Serial Data (SDA) line, while GPIO22 functions as the Serial Clock (SCL) line. These pins enable bidirectional communication between the microcontroller and motion sensor.

GPS Module Pin Configuration

The NEO-6M GPS module communicates through HardwareSerial2 of the ESP32. GPIO16 is configured as the RX2 pin to receive GPS data, while GPIO17 is configured as the TX2 pin for serial transmission.

MQ135 Sensor Pin Configuration

The analog output of the MQ135 sensor is connected to GPIO34, which functions as an ADC input channel. The ESP32 continuously samples this analog signal and converts it into digital values for gas concentration analysis.

GPIO Mapping Table

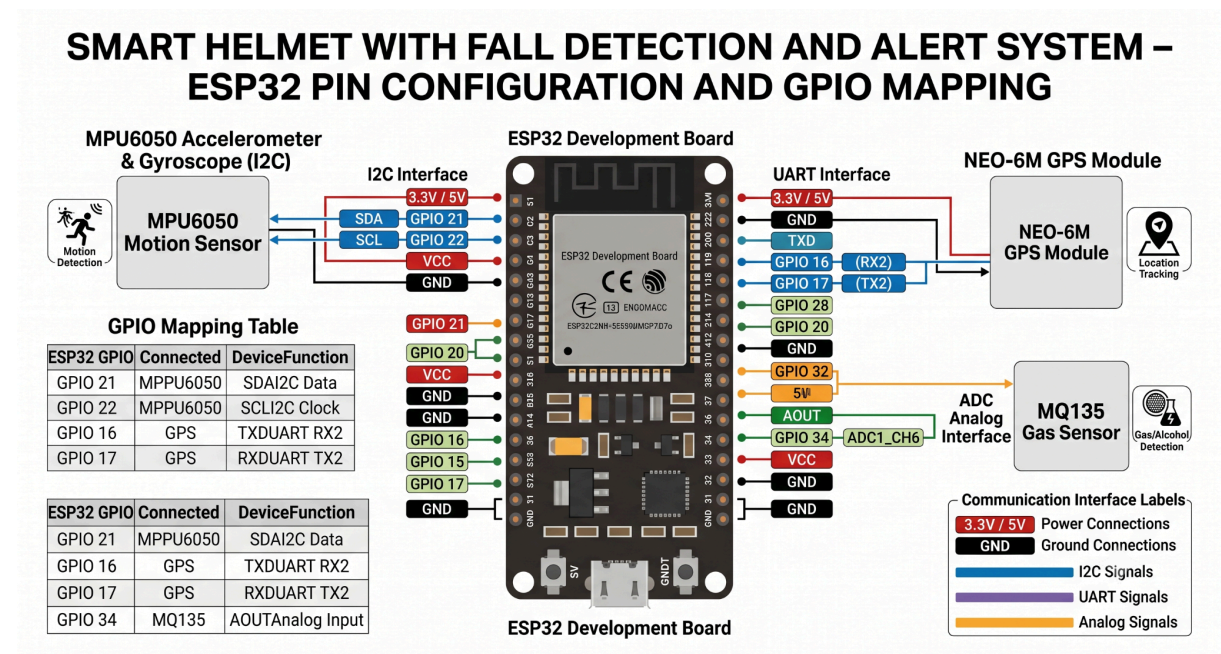


Figure 3.3: ESP32 Pin Configuration and GPIO Mapping

Advantages of the Selected Pin Configuration

- Efficient utilization of dedicated hardware communication peripherals.
- Reduced software overhead through hardware-based communication.
- Improved reliability of sensor data acquisition.
- Simplified circuit implementation and troubleshooting.
- Scalability for future sensor additions.

3.5 Data Flow and System Operation

The operation of the system begins with continuous acquisition of sensor data. The MPU6050 measures acceleration and angular motion, the GPS module provides location coordinates, and the MQ135 sensor measures gas concentration levels.

The ESP32 periodically collects readings from all sensors and calculates the resultant acceleration magnitude. If the measured acceleration exceeds the predefined threshold, the system identifies a potential fall or accident condition. The accident state is maintained using hysteresis logic to prevent false triggering caused by transient sensor fluctuations.

Once an accident is confirmed, telemetry data including GPS coordinates is transmitted to the backend server. The server processes the information and activates the emergency alert mechanism. SMS notifications containing accident details and location information are sent to predefined emergency contacts. Simultaneously, the web dashboard displays visual and audio alerts to notify operators of the incident.

This real-time workflow significantly reduces emergency response time and improves the probability of timely medical assistance.

Chapter 4

Methodology

4.1 Overview

The Smart Helmet with Fall Detection and Alert System is designed using an Internet of Things (IoT) architecture that combines sensing, data processing, wireless communication, and emergency notification mechanisms. The methodology focuses on continuously monitoring rider conditions, detecting fall or accident events, transmitting real-time telemetry data, and generating emergency alerts whenever required.

The system consists of three major components:

1. Smart Helmet Hardware Module
2. Backend Server
3. Web-Based Monitoring Dashboard

The hardware module collects data from multiple sensors and transmits the information to the backend server through Wi-Fi communication. The server processes the received data, manages emergency notifications, and forwards telemetry information to the monitoring dashboard. The dashboard provides real-time visualization of rider information and accident alerts.

4.2 System Architecture

The proposed system follows a three-layer architecture consisting of the sensing layer, communication layer, and application layer.

4.2.1 Sensing Layer

The sensing layer is responsible for collecting physical data from the rider and surrounding environment. It consists of:

- ESP32 Microcontroller
- MPU6050 Accelerometer and Gyroscope
- NEO-6M GPS Module
- MQ135 Gas Sensor

The sensors continuously generate data related to rider movement, location, and environmental conditions.

4.2.2 Communication Layer

The communication layer establishes data exchange between the smart helmet and the backend server using Wi-Fi and WebSocket protocols.

Functions of the communication layer include:

- Real-time telemetry transmission
- Remote command handling
- Data buffering during network failures
- Automatic reconnection management

4.2.3 Application Layer

The application layer consists of:

- Node.js Backend Server
- Emergency Alert System
- Web Dashboard

This layer processes incoming telemetry data, detects emergency situations, sends notifications, and provides live monitoring facilities.

4.3 Hardware Methodology

4.3.1 ESP32 Microcontroller

The ESP32 serves as the central processing unit of the smart helmet. It interfaces with all sensors, processes incoming data, and manages wireless communication with the server.

The ESP32 performs the following tasks:

- Sensor data acquisition
- Accident detection calculations
- GPS data processing
- Telemetry packet generation
- Wi-Fi communication
- WebSocket connection management

Its built-in Wi-Fi capability eliminates the need for additional communication modules, making the system compact and cost-effective.

4.3.2 MPU6050 Motion Sensor

The MPU6050 contains a three-axis accelerometer and a three-axis gyroscope. It continuously measures the motion characteristics of the rider.

The accelerometer provides:

- X-axis acceleration
- Y-axis acceleration
- Z-axis acceleration

The gyroscope provides:

- Angular velocity along X-axis

- Angular velocity along Y-axis
- Angular velocity along Z-axis

These parameters are used to identify sudden impacts and abnormal movements associated with accidents.

4.3.3 GPS Module

The NEO-6M GPS module receives satellite signals and calculates the geographical coordinates of the rider.

The GPS module provides:

- Latitude
- Longitude
- Speed
- Satellite lock status

Location information is essential for emergency response and rider tracking.

4.3.4 MQ135 Gas Sensor

The MQ135 sensor measures gas concentration in the surrounding environment.

The sensor is used to:

- Monitor air quality
- Detect alcohol-related vapors
- Identify potentially harmful environmental conditions

The sensor readings are converted into parts per million (PPM) values for monitoring purposes.

4.4 Accident Detection Methodology

Accident detection is the most important functionality of the proposed system. The detection mechanism is based on continuous monitoring of acceleration values obtained from the MPU6050 sensor.

4.4.1 Data Acquisition

The accelerometer continuously generates acceleration values along three axes:

- A_x
- A_y
- A_z

These values represent the force experienced by the helmet in different directions.

4.4.2 Resultant Acceleration Calculation

The total acceleration experienced by the rider is calculated using vector magnitude.

The acceleration magnitude is given by:

$$A_{total} = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

Where:

- (A_x) = Acceleration along X-axis
- (A_y) = Acceleration along Y-axis
- (A_z) = Acceleration along Z-axis

This calculation provides the overall force acting on the helmet irrespective of direction.

4.4.3 Threshold-Based Detection

A predefined threshold value is used to identify accident conditions.

When:

$$A_{total} = 4.5G$$

the system assumes that a severe impact or fall has occurred.

The accident state is immediately activated and transmitted to the backend server.

4.4.4 Hysteresis Mechanism

To avoid false alarms caused by temporary fluctuations, the system incorporates an 8-second hysteresis mechanism.

The accident state remains active for a specified duration even if acceleration values return to normal levels. This ensures reliable accident detection and prevents repeated triggering.

4.5 GPS Tracking Methodology

The GPS tracking mechanism continuously monitors the rider's position.

Step 1: Satellite Signal Reception

The GPS module receives signals from available satellites.

Step 2: Coordinate Calculation

Using triangulation techniques, the GPS module calculates:

- Latitude
- Longitude

Step 3: Data Validation

The system verifies whether the GPS data is valid.

Step 4: Telemetry Integration

Validated coordinates are attached to telemetry packets.

Step 5: Dashboard Visualization

The server forwards location data to the dashboard where the rider position is displayed on a digital map.

Step 6: Emergency Location Sharing

When an accident occurs, the latest coordinates are included in the emergency notification message.

4.6 Telemetry Communication Methodology

The communication system uses WebSocket technology for real-time data exchange.

4.6.1 Telemetry Generation

The ESP32 creates telemetry packets containing:

- Device ID
- Accident status
- GPS coordinates
- Speed
- Gas concentration
- Accelerometer values
- Gyroscope values

4.6.2 Data Transmission

The telemetry packet is transmitted to the backend server through an active WebSocket connection.

4.6.3 Data Reception

The server receives and validates incoming telemetry data.

4.6.4 Dashboard Broadcasting

The processed data is broadcast to all connected dashboard clients.

This mechanism enables real-time monitoring with minimal communication delay.

4.7 Offline Data Buffering Methodology

Network interruptions are common in mobile environments. To ensure reliable operation, the system includes an offline buffering mechanism.

Data Storage

When Wi-Fi or server connectivity is unavailable:

- Telemetry packets are stored locally.
- A circular buffer with fixed capacity is used.

Data Recovery

Once communication is restored:

- Buffered packets are transmitted sequentially.
- Delays are introduced between transmissions to prevent server overload.

Advantages

- Prevents data loss.
- Maintains telemetry continuity.
- Improves system reliability.

4.8 Emergency Alert Methodology

The emergency alert subsystem is activated whenever an accident is detected.

Step 1: Accident Detection

The ESP32 identifies an accident event using acceleration analysis.

Step 2: Alert Transmission

The accident status is transmitted to the backend server.

Step 3: Location Retrieval

The server retrieves the latest GPS coordinates.

Step 4: SMS Generation

An emergency message is generated containing:

- Accident warning
- Latitude and longitude
- Google Maps location link

Step 5: SMS Dispatch

The Twilio API is used to send SMS notifications to predefined emergency contacts.

Step 6: Rate Limiting

A cooldown period is implemented to prevent repeated message transmission for the same incident.

4.9 Dashboard Monitoring Methodology

The web dashboard serves as the user interface of the system.

Real-Time Location Monitoring

The dashboard displays:

- Rider location
- GPS status
- Interactive map

Motion Data Visualization

Acceleration values are plotted using graphical charts for continuous monitoring.

Gas Monitoring

Gas concentration levels are displayed using visual indicators.

Accident Alerts

When an accident occurs:

- Alert overlays are displayed.
- Audio sirens are activated.
- Emergency status indicators become visible.

Alarm Control

Authorized users can dismiss alerts, which resets the accident state throughout the system.

4.10 System Workflow

The overall workflow of the proposed system is summarized below:

- Sensors continuously collect rider and environmental data.
- ESP32 processes the acquired information.
- Acceleration values are analyzed for accident detection.
- GPS coordinates and sensor readings are packaged into telemetry frames.
- Telemetry data is transmitted to the backend server.
- The server forwards data to the monitoring dashboard.
- If an accident is detected, emergency alerts are generated automatically.
- SMS notifications containing location information are sent to emergency contacts.
- The dashboard displays visual and audio alerts for real-time monitoring.
- The system continues operation while maintaining communication reliability through buffering and automatic reconnection mechanisms.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Introduction

The Smart Helmet with Fall Detection and Alert System was successfully designed and implemented using an ESP32 microcontroller, MPU6050 motion sensor, NEO-6M GPS module, and MQ135 gas/alcohol sensor. The system was tested under different operating conditions to evaluate its performance in accident detection, location tracking, alcohol detection, real-time monitoring, and emergency alert generation. The obtained results demonstrate the effectiveness of the proposed system in enhancing rider safety and providing timely emergency assistance.

5.2 Fall Detection Results

The MPU6050 accelerometer and gyroscope were used to monitor rider movement continuously. During normal riding conditions, the measured acceleration values remained within the predefined safe range. To test accident detection, sudden impacts and fall scenarios were simulated.

The system successfully detected abrupt acceleration changes exceeding the configured threshold and classified them as accident events. Upon detection, the accident flag was activated and transmitted to the backend server. The alert state remained active through hysteresis control, preventing false triggering caused by temporary vibrations or sensor fluctuations.

The experimental results showed that the system was capable of detecting fall events accurately and responding immediately to emergency situations.

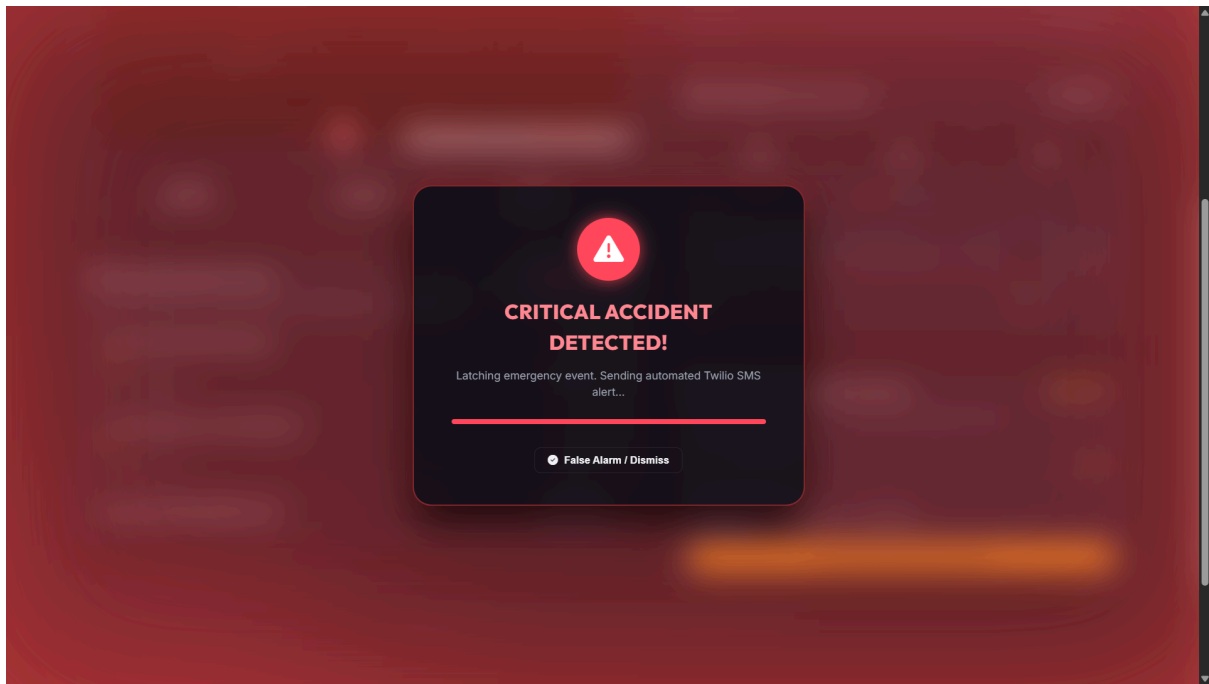


Figure 5.3: Accident Detection Alert

5.3 GPS Tracking Results

The NEO-6M GPS module continuously provided real-time geographical coordinates of the rider. The location data was successfully transmitted to the backend server and displayed on the monitoring dashboard.

The GPS module was able to:

- Obtain latitude and longitude coordinates.
- Measure rider speed.
- Monitor satellite connectivity status.
- Update rider location in real time on the dashboard map.

During testing, the location information was accurately plotted on the map interface. In accident scenarios, the current coordinates were automatically included in emergency alerts, allowing emergency contacts to identify the rider's location quickly.

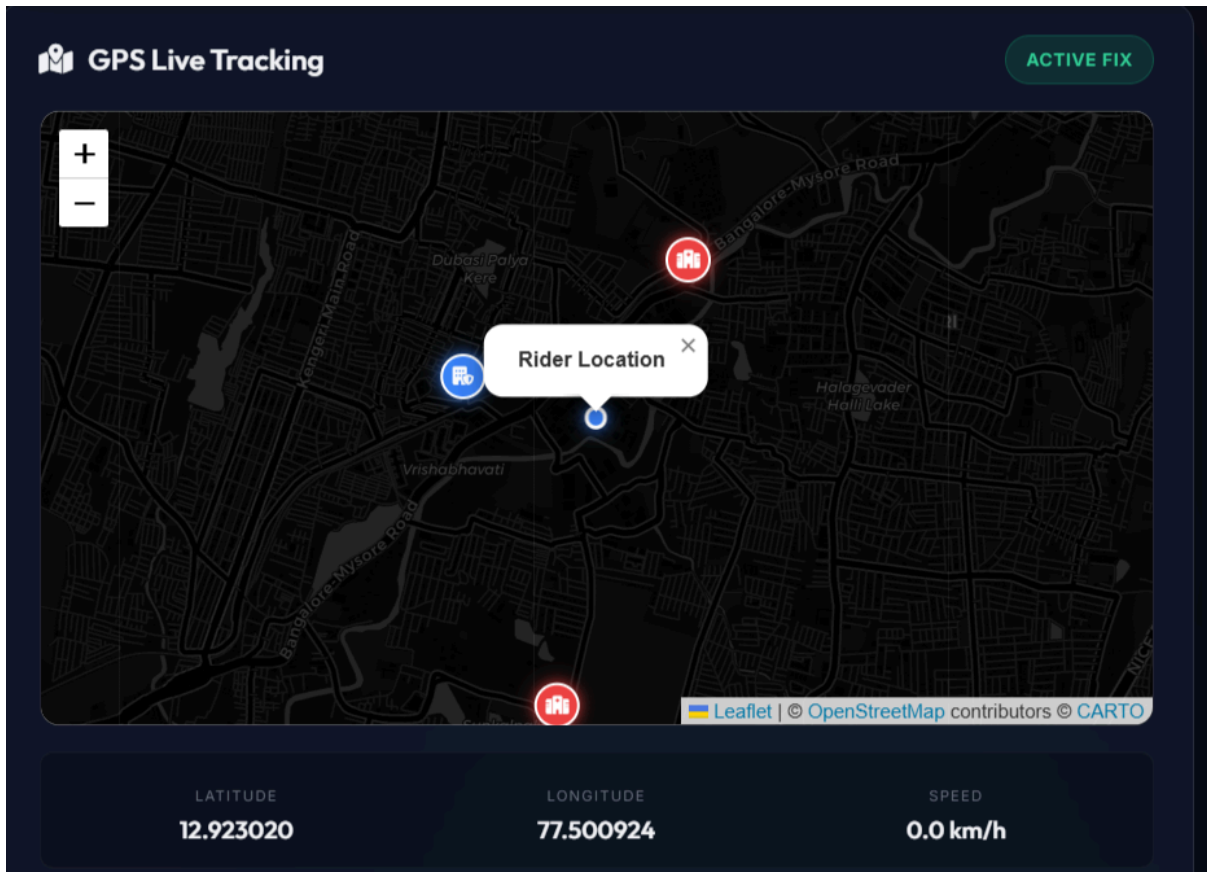


Figure 5.4: GPS Location Tracking

5.4 Alcohol Detection Results

The MQ135 sensor was utilized for alcohol and gas detection. The sensor continuously monitored the surrounding air and generated analog output values corresponding to the detected concentration levels.

Several test scenarios were conducted using alcohol-based substances near the sensor. The observed readings increased significantly in the presence of alcohol vapors compared to normal environmental conditions.

The dashboard categorized the readings into different safety levels:

- Clean Air Condition
- Moderate Gas Presence
- High Alcohol/Gas Detection
- Critical Alert Condition

The sensor successfully detected variations in alcohol concentration and generated warning indications whenever the measured values exceeded predefined thresholds. This feature helps identify potential alcohol influence and promotes safer riding practices.

The experimental results confirmed that the MQ135 sensor can effectively function as an alcohol detection component within the proposed safety system.

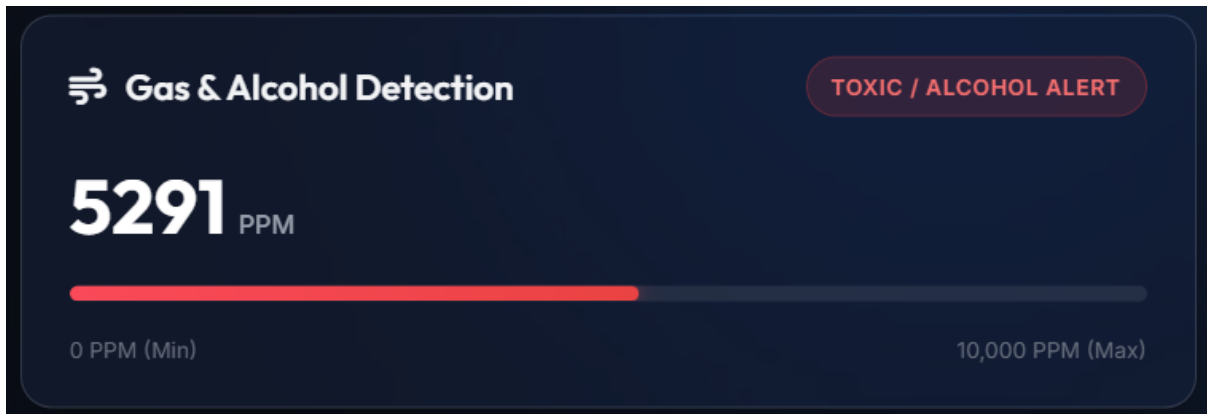


Figure 5.5: Alcohol Detection Monitoring

5.5 Emergency Alert Results

One of the primary objectives of the project was to provide immediate emergency notification after accident detection.

When an accident event occurred:

1. The ESP32 detected the abnormal acceleration.
2. GPS coordinates were acquired.
3. Telemetry data was transmitted to the server.
4. The backend generated an emergency alert.
5. SMS notifications were sent to registered emergency contacts.

The alert message included:

- Accident notification.
- GPS coordinates.
- Google Maps location link.
- Emergency warning information.

The alert delivery process was completed within a few seconds after accident detection, demonstrating the effectiveness of the emergency response mechanism.

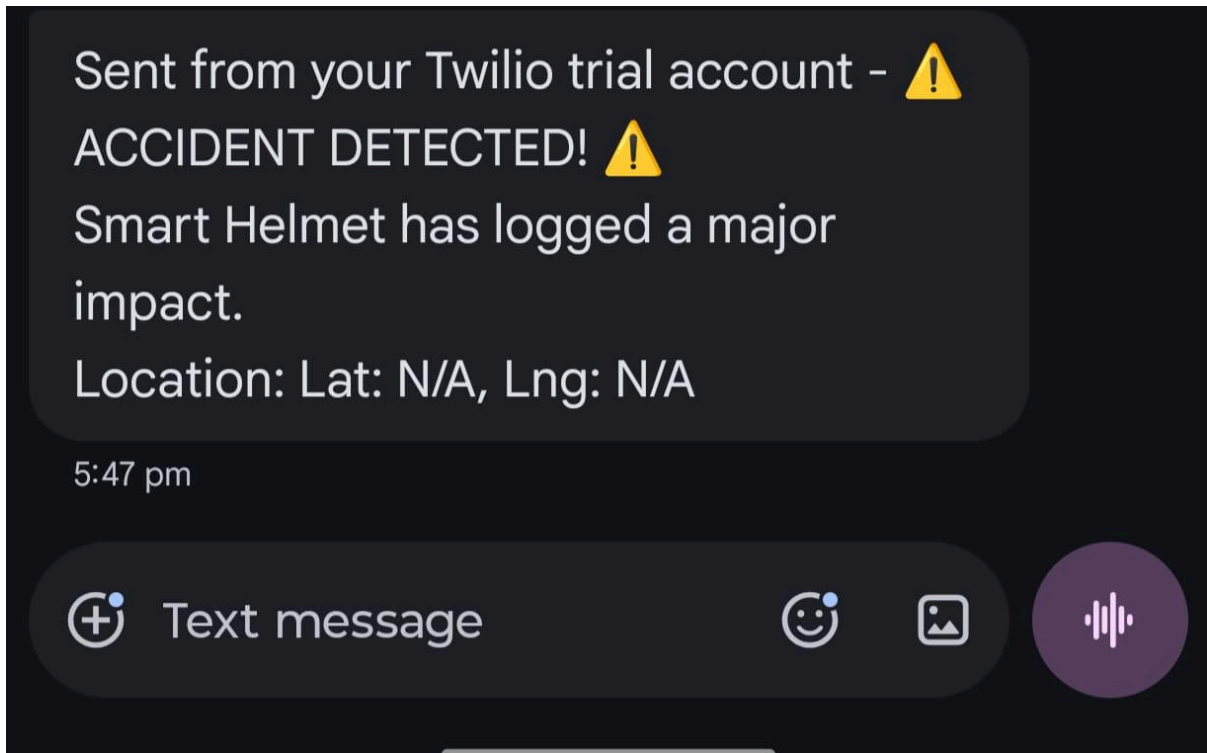


Figure 5.5: Emergency SMS Alert Received

5.6 Dashboard Monitoring Results

The web dashboard successfully displayed real-time telemetry data received from the ESP32 device.

The dashboard provided:

- Live GPS tracking.
- Real-time acceleration monitoring.
- Alcohol/gas concentration monitoring.
- Accident status indication.
- Audio and visual emergency alerts.

The graphical representation of sensor data improved system usability and enabled continuous remote monitoring. During testing, the dashboard responded immediately to sensor updates and accurately reflected changes in system status.

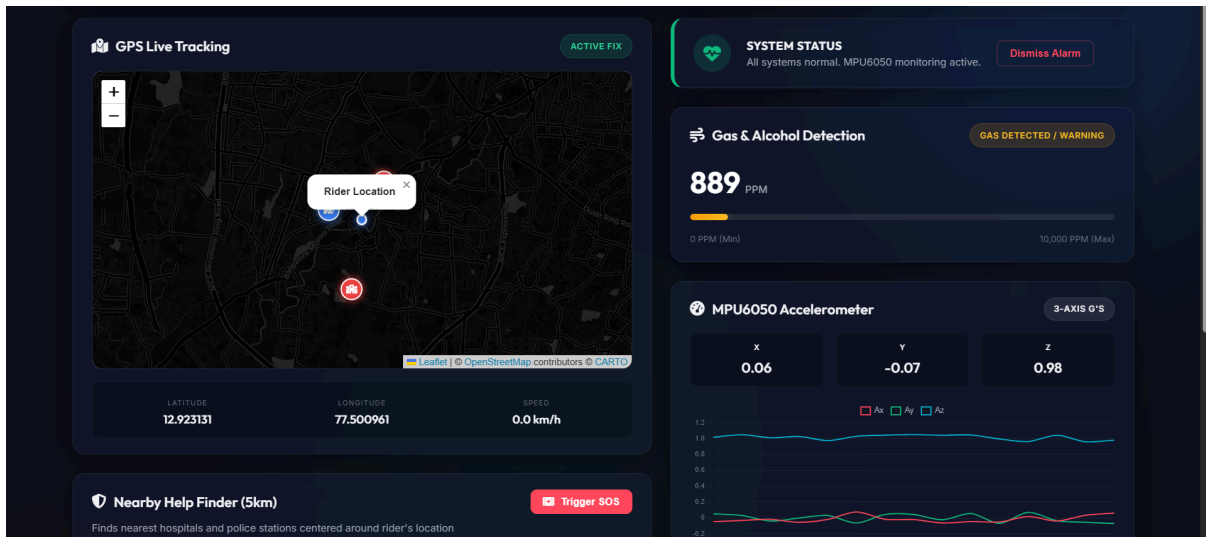


Figure 5.6: Dashboard Home Screen

5.7 System Performance Analysis

The integrated system operated reliably throughout the testing period. The use of WebSocket communication enabled low-latency data transfer between the helmet and server. The implemented buffering mechanism prevented data loss during temporary network interruptions.

Additional reliability features such as GPS staleness detection, automatic reconnection, and accident-state latching improved overall system stability.

The developed prototype achieved the following objectives:

- Accurate fall detection using motion sensing.
- Real-time GPS tracking.
- Alcohol detection using MQ135 sensor.
- Automatic emergency alert generation.
- Live dashboard monitoring.
- Reliable wireless communication.

5.8 Discussion

The obtained results demonstrate that the Smart Helmet with Fall Detection and Alert System can significantly improve rider safety by combining accident detection, alcohol monitoring, GPS tracking, and emergency communication into a single platform. The integration of IoT technologies enables continuous monitoring and rapid response during emergencies.

The system successfully fulfilled its intended objectives and provides a practical solution for reducing emergency response time following road accidents. Future enhancements such as cloud integration, mobile applications, machine learning-based accident prediction, and advanced health monitoring sensors can further improve system capabilities.

APPENDIX

Appendix A: Source Code

The complete source code developed for the Smart Helmet with Fall Detection and Alert System consists of the following modules:

A.1 ESP32 Firmware

The ESP32 firmware is responsible for:

- Reading MPU6050 accelerometer and gyroscope data.
- Acquiring GPS coordinates from the NEO-6M module.
- Monitoring alcohol and gas concentration using the MQ135 sensor.
- Detecting fall and accident events.
- Establishing Wi-Fi connectivity.
- Transmitting telemetry data to the backend server through WebSocket communication.
- Receiving control commands from the server.

A.2 Backend Server Code

The Node.js backend server performs:

- Real-time communication with ESP32 devices.
- WebSocket management.
- Emergency alert processing.
- GPS location handling.
- SMS notification generation using Twilio.
- Dashboard data broadcasting.

A.3 Web Dashboard Code

The web dashboard provides:

- Live telemetry monitoring.
- Real-time GPS tracking.
- Accident alert visualization.
- Gas and alcohol monitoring.
- User interaction and control features.

Appendix B: Acronyms

Acronym	Full Form
ADC	Analog-to-Digital Converter
API	Application Programming Interface
ESP32	Espressif 32-bit Microcontroller
GPS	Global Positioning System
GND	Ground
GPIO	General Purpose Input/Output
GSM	Global System for Mobile Communications
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
I2C	Inter-Integrated Circuit
IMU	Inertial Measurement Unit
IoT	Internet of Things
JSON	JavaScript Object Notation
LCD	Liquid Crystal Display
MPU6050	Motion Processing Unit 6050
MQTT	Message Queuing Telemetry Transport
NEO-6M	GPS Receiver Module
Node.js	JavaScript Runtime Environment
OSM	OpenStreetMap
PCB	Printed Circuit Board

PPM	Parts Per Million
REST	Representational State Transfer
RX	Receive
SDA	Serial Data Line
SCL	Serial Clock Line
SMS	Short Message Service
SPI	Serial Peripheral Interface
TX	Transmit
UART	Universal Asynchronous Receiver-Transmitter
URL	Uniform Resource Locator
USB	Universal Serial Bus
Wi-Fi	Wireless Fidelity
WS	WebSocket

Appendix C: Hardware Specifications

ESP32 Development Board

- Dual-core 32-bit microcontroller
- Operating Voltage: 3.3V
- Integrated Wi-Fi and Bluetooth
- Multiple GPIO interfaces

MPU6050 Sensor

- 3-axis Accelerometer
- 3-axis Gyroscope
- I2C Communication Interface

NEO-6M GPS Module

- Real-time Location Tracking
- UART Communication Interface
- Latitude and Longitude Acquisition

MQ135 Gas Sensor

- Alcohol Detection
- Air Quality Monitoring
- Analog Output Interface

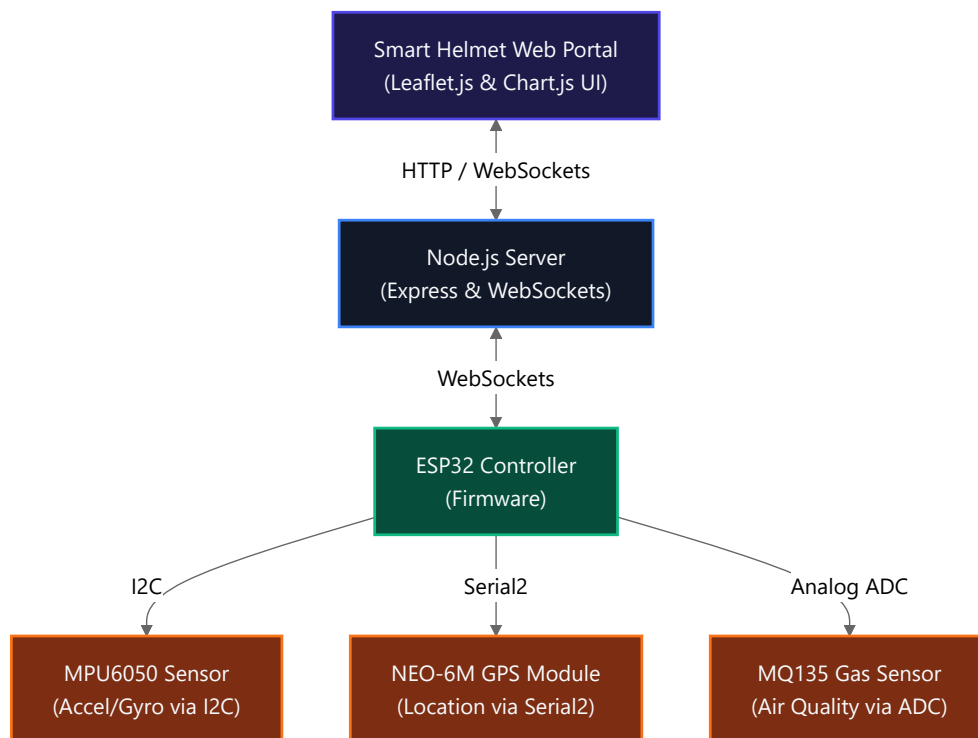
Appendix D: Software Tools Used

Software Tool	Purpose
Arduino IDE	ESP32 Firmware Development
Visual Studio Code	Backend Development
Node.js	Server Runtime Environment
Express.js	Backend Framework
Chart.js	Data Visualization
Leaflet.js	GPS Map Visualization
Twilio API	SMS Alert Service
GitHub	Version Control
WebSocket	Real-Time Communication

Smart Helmet with Fall Detection and Alert System

Smart Helmet with Fall Detection and Alert System is an advanced, IoT-enabled safety and telemetry system designed to protect riders. Powered by an ESP32 microcontroller, a Node.js backend, and a real-time web-based dashboard, it monitors physical parameters (acceleration, rotation, GPS coordinates, air quality/gas concentration) to detect accidents in real-time, automatically dispatch emergency SMS alerts, and assist in locating nearby emergency services.

Project Architecture



The system comprises three core components:

1. **ESP32 Firmware** (`/esp32_firmware`): Interfaces with physical sensors, runs edge accident-detection algorithms, manages connections with fallback logic, and buffers telemetry locally during network drops.
2. **Node.js Backend** (`/server`): Hosts the Express server, manages state routing, handles WebSocket handshakes from the helmet and the dashboard, interacts with the Twilio SMS gateway, and coordinates nearby emergency resource queries.

3. **Web Dashboard** (`/server/public`): Built with pure CSS, HTML, and vanilla Javascript. Includes interactive Leaflet JS maps, Chart.js real-time IMU graphing, a visual emergency sirens overlay, and system diagnostics panels.
-

Key Features

- **Real-time Telemetry Streaming:** Instant WebSocket data synchronization between the ESP32 helmet unit, node server, and dashboard (2s update interval).
 - **Edge Accident Detection:** The ESP32 monitors the MPU6050 accelerometer. A sudden impact matching or exceeding **4.5 G's** triggers an immediate hardware interrupt, bypassing standard timers to dispatch a critical accident alarm.
 - **Hysteresis & Alarm Latching:** Accident alerts latch for **8 seconds** to ensure warning alerts persist while monitoring post-impact forces. The alarm can be dismissed remotely from the web dashboard or will auto-clear if forces return to normal.
 - **Offline Telemetry Buffer:** An on-device circular queue buffer stores telemetry frames during cellular or Wi-Fi disconnection. It automatically flushes buffered frames sequentially with a throttle delay once connection is restored.
 - **Dynamic Location Services & Tracking:** Live tracking on an interactive map. If GPS signals are lost, the device flags the coordinates as stale.
 - **Automated Emergency SOS Messaging:** Integrates with Twilio SMS APIs to automatically send a text alert to designated emergency contacts containing exact GPS coordinates and a Google Maps tracking link.
 - **Nearby Emergency Services Finder:** Centered around the rider's position, the backend queries Google Places API (or falls back to OpenStreetMap Overpass API dynamically) to find hospitals, clinics, and police stations within a **5 km** radius, providing names, ratings, addresses, and directions.
 - **Comprehensive Diagnostics Panel:** Dashboard contains controls to adjust location hops (Bangalore, New York, London), slide mock alcohol/gas concentrations, and fire a high-g collision event, allowing complete system test runs without active hardware.
-

Hardware Setup & Pin Mapping

Connect the sensors to your ESP32 board according to the following configurations:

Component	Sensor Pin	ESP32 GPIO Pin	Description
MPU6050 IMU	VCC	3.3V	3.3V Power Input
	GND	GND	Ground
	SDA	GPIO 21	I2C Data Line
	SCL	GPIO 22	I2C Clock Line
NEO-6M GPS	VCC	3.3V / 5V	Power Input
	GND	GND	Ground
	TXD	GPIO 16 (RX2)	Hardware Serial 2 Receive
	RXD	GPIO 17 (TX2)	Hardware Serial 2 Transmit
MQ135 Gas Sensor	VCC	5V	5V Power Input
	GND	GND	Ground
	AOUT	GPIO 34 (ADC1)	Analog Signal Output

Required Arduino Libraries

Before flashing the ESP32 code, install the following libraries in your Arduino IDE (`Tools -> Manage Libraries...`):

1. **Adafruit MPU6050** (by Adafruit)
2. **Adafruit Unified Sensor** (by Adafruit)
3. **TinyGPS++** (by Mikal Hart)
4. **WebSockets** (by Markus Sattler)
5. **ArduinoJson** (by Benoit Blanchon)

Setup & Execution Guide

1. Backend Server Installation

1. Navigate to the server folder:

```
cd server
```

2. Install npm dependencies:

```
npm install
```

3. Configure the environment variables:

- Copy the sample configuration file:

```
copy .env.example .env
```

- Open the new `.env` file and input your credentials:
 - `PORT` : Port to host server (port: `3000`).
 - `TWILIO_ACCOUNT_SID` , `TWILIO_AUTH_TOKEN` , `TWILIO_FROM_NUMBER` , `TWILIO_TO_NUMBER` : Required for SMS alerts. *Leave blank to log SMS content to the console.*
 - `GOOGLE_PLACES_API_KEY` : Used to locate emergency facilities. *Leave blank to use OpenStreetMap Overpass fallback.*

4. Launch the server:

```
npm start
```

The server runs at <http://localhost:3000> (<http://localhost:3000>).

2. ESP32 Firmware Configuration & Flashing

1. Open `esp32_firmware/esp32_firmware.ino` in your Arduino IDE.
2. Modify the configuration constants at the top of the file:
 - `ssid` : Your Wi-Fi SSID network name.
 - `password` : Your Wi-Fi network password.
 - `server_host` : Your local machine's IP address (run `ipconfig` on Windows or `ifconfig` on Linux/macOS to locate it. Ensure the ESP32 and host server are connected to the same network segment).
 - `server_port` : Set to match your backend port (port: `3000`).

3. Connect your ESP32 board to your computer via USB.
 4. Select the appropriate board model (`Tools -> Board -> ESP32 Dev Module`) and communication port.
 5. Click **Upload** to compile and transfer the program.
 6. Open the **Serial Monitor** (Baud Rate: `115200`) to observe connectivity logs, sensor statuses, and diagnostics.
-



Dashboard Operations & Walkthrough

The web dashboard is fully dynamic and automatically adjusts styling depending on connection state:

- **Siren Visual Overlay:** Triggering an accident (either via hardware shock or diagnostics panel UI) activates a blinking full-screen warning overlay with an alarm siren, triggering Twilio SMS dispatch, and kicking off nearby services searching.
- **IMU Real-time Chart:** Accelerometer readouts are plotted on a rolling line chart to monitor physical forces dynamically.
- **System Diagnostics:**
 - **Gas/Alcohol Slider:** Slide to test high alcohol or toxic gas values. Levels above **1,000 PPM** switch the air quality badge warning to yellow/red.
 - **Location Presets:** Toggle buttons to snap coordinates between Bangalore, New York, or London to test spatial API queries.
 - **Trigger Impact Button:** Instantly triggers a test crash vector, testing warning triggers and API fallbacks.
 - **False Alarm/Dismiss button:** Allows clearing active accident states, updating the hardware through server-side back-propagation.